Actividad 9. Esquema Algorítmico Vuelta Atrás

Objetivo

Resolver problemas que siguen el esquema algorítmico de vuelta atrás.

Procedimiento

1. Ver el vídeo o leer la presentación sobre esquemas algorítmicos que están disponibles en Moodle, Tema 4/Sección 4.2 Esquema Algorítmico: Vuelta atrás/ Recursos didácticos.
2. Resolver los ejercicios que se proponen en esta actividad, para reforzar el esquema de vuelta atrás. Para ello haz uso de la clase BacktrackingAlgorithm situada en el paquete **es.uvigo.esei.aed2.activity9**, la cual contiene las cabeceras de los métodos.
3. Para probar que la implementación realizada funciona correctamente se proporcionan la clase de prueba BacktrackingAlgorithmTestCase.

Evaluación

Estos contenidos serán evaluados mediante una prueba individual el 19 de enero de 2026.

Tiempo estimado

3 horas y media.

Ejercicios para resolver en clase

1. Problema de dar el cambio. Supongamos que el cajero sólo tiene billetes de 200 y 500 euros y nos debe dar 2100 euros. Con una estrategia voraz nunca llegaría a la solución correcta (daría 4 de 500 y no podría dar el resto), mientras que con vuelta atrás podemos ir dando billetes y si llegamos a una combinación sin solución, volvemos atrás intentándolo con otro billete y así sucesivamente. En este caso la solución sería 3 billetes de 500 y 3 de 200. Implementa este algoritmo siguiendo una estrategia de **vuelta atrás**.

   ```
   public static boolean giveChange(int amountReturned, Map<Integer, Integer>
   changeAvailable, Map<Integer, Integer> solution)
   ```

2. Considere que tenemos n programas distintos para grabar en un pendrive, pero el espacio de memoria que necesitan excede la capacidad del dispositivo. Cada programa P_i requiere m_i kilobytes de memoria, la capacidad del pendrive es de C kilobytes y $C < \sum_{i=1}^{n} m_i$

Resuelve este algoritmo siguiendo un esquema de **vuelta atrás**, de tal forma que si existe solución consigue ocupar el espacio del pendrive completamente, sino lo devuelve vacío.

El método principal tendrá esta cabecera:

```
public static boolean fillUpFlashDrive(int capacityCD, Map<String, Integer> espacePrograms, Set<String> flashDrive)
```

donde:

CapacityCD: es la capacidad del pendrive

espacePrograms: es un Map que contiene el nombre de cada programa disponible (identificado por un String) y el espacio que ocupa (Integer).

flashDrive: es un conjunto que contiene el nombre de los programas grabados, inicialmente estará vacío.

3. Considérese un "vector" que almacena n enteros positivos distintos: vector = $\{v_1, v_2, v_3, \ldots, v_n\}$. El problema de la suma consiste en encontrar un subconjunto del vector cuya suma sea exactamente "resultado".

   La "solución" tendrá la forma de vector con valores 1 y 0, es decir, solución = $(s1, s_2, s_3, \ldots, s_n\}$ con $s_i \in \{0,1\}$ y deberá cumplir: $\sum v_i * s_i$ = resultado.

   Para resolver este problema deberá utilizarse la estrategia de **vuelta atrás**. Que irá considerando todas las posibilidades (incluir / no incluir cada elemento en la solución).

   El método que resuelve el problema tendrá la siguiente cabecera:

```
public static boolean subgroups(int[] values, int[] solution, int result, int index)
```

   siendo:

   values: el vector con n enteros positivos distintos

   solution: el vector de 0 y 1 con la solución al problema

   result: lo que tiene que sumar los enteros seleccionados del vector.

   index: posición del vector a considerar incluir o no incluir en la solución. Puede considerarse que vale 0 la primera vez que se invoca al método.

4. Problema de las 8 reinas. El problema consiste en colocar ocho reinas en un tablero de ajedrez sin que se den jaque (dos reinas se dan jaque si comparten fila, columna o diagonal). Puesto que no puede haber más de una reina por fila, podemos replantear el problema como "colocar una reina en cada fila del tablero de forma que no se den jaque". En este caso, para ver si dos reinas se dan jaque basta con ver si comparten columna o diagonal (emplea el método privado *goodPlace*). Por lo tanto, toda solución del problema se puede representar como una 8-tupla $(X_0,\ldots,X_7)$ en la que X_i es la columna en la que se coloca la reina que está en la fila i del tablero. Implementa el algoritmo *placeQueens*, siguiendo una estrategia de **vuelta atrás** para resolver el ejercicio.

```
public static boolean placeQueens (int queen, int[] board)
```

5. El Sudoku se ha convertido en el pasatiempo de moda. Se trata de una matriz de 9x9 celdas, donde en cada celda puede haber un número entre 1 y 9.

   Se añade la restricción de que cada número sólo puede aparecer una vez en cada fila, en cada columna y en cada bloque de celdas. Los bloques son grupos de 3x3 celdas adyacentes, como se muestran abajo (ver que, en total, hay 9 filas, 9 columnas y 9 bloques). Algunas celdas del Sudoku están inicializadas con ciertos valores y otras están vacías. El objetivo es completar las celdas vacías, con valores que cumplan las restricciones del juego.

   Se pide diseñar un algoritmo de **vuelta atrás** para intentar resolver el problema. Suponer que la entrada es una matriz de 9x9, donde un valor 0 indica una celda no inicializada.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

   Dado el método **isPossibleInsert**, implementa el método **solveSudoku**, el cual modifica el tablero con la solución al Sudoku.

```
public static boolean solveSudoku(int[][] board)
```

6. Implementar el método que se especifica más abajo, para que encuentre la forma de recorrer un laberinto n * n desde la casilla de salida (0,0) hasta la casilla de llegada (n-1, n-1). Cada casilla de la matriz está etiquetada con una letra, de forma que en el camino desde la salida hacia la llegada se debe ir formando (de forma cíclica) una palabra dada. Sigue el esquema de **Vuelta Atrás**.

```
public static boolean rehearseWords(char[][] maze, int positionX, int
positionY, String word, int pos)
```

   Donde:

   maze: es la matriz n * n, con letras.

   positionX: posición X de la matriz.

   positionY: posición Y de la matriz.

   word: la cadena que se debe ir formando por el camino de la matriz.

   pos: la posición de la cadena que contiene la letra que debe contener la matriz en la posición (positionX, positionY).

Desde una celda se puede ir a cualquiera de las cuatro celdas adyacentes; para marcar el camino, si coincide con la letra buscada se debe modificar por un ‘ ‘. A continuación, aparece un ejemplo en el que la palabra que hay que ir formando por el camino es EDA, comenzando en la casilla (0,0) y finalizando en (4,4).

Matriz inicial:

	0	1	2	3	4
0	E	D	A	O	S
1	X	E	E	R	C
2	U	J	D	V	B
3	Y	K	A	E	D
4	M	T	P	X	A

Matriz resultado:

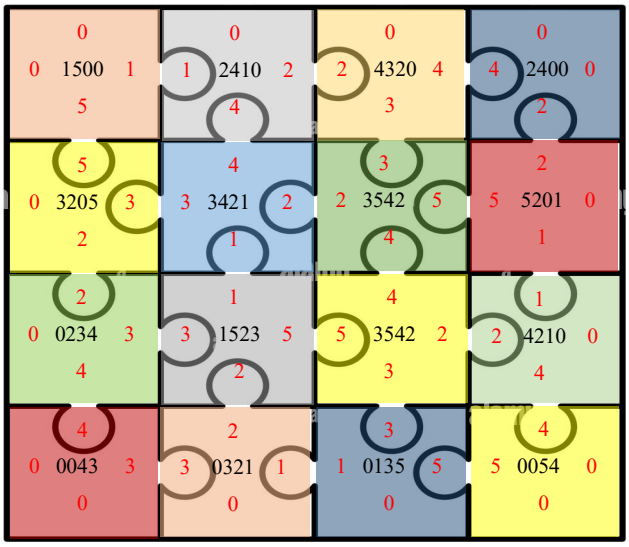
	0	1	2	3	4
0				O	S
1	X	E		R	C
2	U	J		V	B
3	Y	K			
4	M	T	P	X	

7. Dadas unas piezas de puzzle, cada una con 4 lados y cada lado representado por un número entero positivo de 0 a 9, implementa un método que resuelva el rompecabezas (se puede suponer que el tablero va a ser cuadrado). En las fichas el número 0 representa el borde y las piezas no pueden tener dos lados iguales (es decir, no pueden tener el mismo número repetido dos veces, excepto las piezas que ocuparán las esquinas del puzzle que tendrán repetido el borde 0, puedes suponer que la cadena contendrá la subcadena “00”). Los lados de las piezas se presentan secuencialmente en el sentido de las agujas del reloj, empezando por el lado izquierdo. Considera que las **fichas pueden girarse**.

   Se pide diseñar un algoritmo de **vuelta atrás** para intentar resolver el problema, considerando que dos piezas encajan cuando coincide el número de los lados que se tocan. A continuación, se muestra un ejemplo.

   Se dispone de 16 fichas identificados sus lados por una cadena de cuatro números, por ejemplo: "0321", "0135", "0054", ”1523", "2354", "3421", "2410", "3205", ”1500", "4320", "2400", "0043", "3542", "4210", "5201", "0234".

   Una posible solución del puzzle, sería colocar las fichas de la siguiente forma (en el centro y en color negro se representa la etiqueta de la ficha, en rojo el número de cada lado y cómo estaría colocado. Fíjate que algunas fichas están giradas):



Desde un punto de vista de implementación, se puede pensar en utilizar una **matriz** 4x4 para representar el tablero, por lo tanto, las fichas anteriores se almacenarían de la siguiente forma:

“1500”	“2410”	“4320”	“2400”
“3205”	“3421”	“3542”	“5201”
“0234”	“1523”	“3542”	“4210”
“0043”	“0321”	“0135”	“0054”

Otra posible solución del puzzle sería colocar las fichas en este orden:

“0054”	“0135”	“0321”	“0043”
“4210”	“2354”	“1523”	“0234”
“5201”	“3542”	“3421”	“3205”
“2400”	“4320”	“2410”	“1500”

Antes de comenzar con la implementación de la solución y suponiendo que las fichas se colocan de izquierda a derecha y de arriba hacia abajo en el tablero, responde a las siguientes preguntas:

a) Cómo vas a representar una ficha? Es suficiente con la cadena que representa los 4 lados? Piensa qué información necesitas cuándo una ficha ya está colocada en el tablero.
b) Cómo se sabe si una ficha se puede colocar en una esquina? Para saber si una ficha encaja, hay que hacer las mismas comprobaciones dependiendo de la esquina en la que se está colocando la ficha?
c) Cómo se sabe si una ficha se puede colocar en un borde? Para saber si una ficha encaja, hay que hacer las mismas comprobaciones si la ficha se coloca en el borde de arriba, en el de la derecha, en el de la izquierda o en el de abajo?
d) Cómo se sabe si una ficha se puede colocar en el centro? Qué comprobaciones hay que realizar para saber si la ficha encaja?
e) Si una ficha que se ha insertado en el tablero, hay que quitarla (vuelta atrás), necesitarás dejar la ficha exactamente igual que antes de utilizarla.

Será necesario un método que deje igual las fichas que encajaban con ella?

Una vez respondidas estas cuestiones, puedes implementar el método **solvePuzzle**, utiliza los métodos privados que se indican a continuación y la clase **PuzzleCard** disponibles en el paquete **es.uvigo.esei.aed2.activity9**. Emplea un esquema de **vuelta atrás**.

```
private static void changeSidesPuzzleCards(PuzzleCard[][] board)

private static boolean isPossibleInsertCorner(PuzzleCard[][] board, int i, int j, PuzzleCard value1)

private static boolean isPossibleInsertEdge(PuzzleCard[][] board, int i, int j, PuzzleCard value1)

private static boolean isPossibleInsertCentre(PuzzleCard[][] board, int i, int j, PuzzleCard value1)

public static boolean solvePuzzle (PuzzleCard[][] board, List<PuzzleCard> cards)
```

8. Problema de atravesar un laberinto. Nos encontramos en una entrada de un laberinto y debemos intentar atravesarlo. Dentro del algoritmo nos encontraremos con muros que no podremos atravesar, sino que habrá que rodear, lo que hará que nos encontremos a veces en un callejón sin salida. Es necesario tener en cuenta las siguientes condiciones:

   - Representación: Array N x N, de casillas marcadas como libre (**indicadas por el carácter ' '**), ocupada por una pared (**indicadas por el carácter 'X'**) o casillas para Teletransportarse (**indicadas por el carácter 'T'**). (Puede asumirse que el laberinto está bien formado y siempre tendrá o dos 'T', o ninguna).
   - Es posible pasar de una casilla a otra moviéndose solamente en vertical u horizontal. O si se encuentra el carácter 'T' se mueve hasta la otra 'T'.
   - El problema se soluciona cuando desde la casilla (0,0) se llega a la casilla (n-1, n-1).
   - Para resolver este problema se diseñará un algoritmo de búsqueda con retroceso de forma que se marcará en la misma matriz del laberinto un camino solución si existe (**el camino se marcará con el carácter 'C'**).
   - Si por el camino recorrido se llega a una casilla desde la que es imposible encontrar una solución, hay que volver atrás y buscar otro camino (**la casilla imposible se marcará con el carácter 'I'**).
   - Por tanto, hay que marcar las casillas por donde ya se ha pasado (**con el carácter 'C' o 'I'**), de esta forma se evita meterse varias veces por el mismo callejón sin salida, dar vueltas alrededor de columnas....

   Emplea una estrategia de **vuelta atrás** para resolver el ejercicio.

```
public static boolean exitMaze(char[][] maze, int positionX, int positionY)
```